ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ОБРАЗОВАНИЮ

ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ «САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ имени академика С.П. КОРОЛЁВА»

КАФЕДРА «ТЕХНИЧЕСКАЯ КИБЕРНЕТИКА»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ

«Объектно-ориентированное программирование»

ЛАБОРАТОРНАЯ РАБОТА №3

Студент Богданчиков М.А.

Группа 6301-030301D

Руководитель Борисов Д. С.

Оценка _____

САМАРА 2025

Цель лабораторной работы: Дополнить пакет для работы с функциями одной переменной, заданными в табличной форме, добавив классы исключений, новый класс функций и базовый интерфейс.

Ход выполнения лабораторной работы

Задание 1: Изучена документация:
java.lang.Exception, java.lang.IndexOutOfBoundsException, java.lang.ArrayIndexOutOfBoundsException, java.lang.IllegalArgumentException, java.lang.IllegalStateException

Задание 2: Созданы 2 класса исключения:
FunctionPointIndexOutOfBoundsException – исключение выхода за границы набора точек при обращении к ним по номеру, наследует от класса IndexOutOfBoundsException;

InappropriateFunctionPointException – исключение, выбрасываемое при попытке добавления или изменения точки функции несоответствующим образом, наследует от класса Exception.

```
package functions;

public class FunctionPointIndexOutOfBoundsException extends IndexOutOfBoundsException {
    public FunctionPointIndexOutOfBoundsException() {
        super();
    }

    public FunctionPointIndexOutOfBoundsException(String message) {
        super(message);
    }
}
```

Код 1.1

```
package functions;

public class InappropriateFunctionPointException extends Exception {
    public InappropriateFunctionPointException() {
        super();
    }

    public InappropriateFunctionPointException(String message) {
        super(message);
    }
}
```

Код 1.2

Задание 3:
Модифицирован класс табулированной функции TabulatedFunction, написанный в прошлой лабораторной работе. Теперь конструкторы и методы выбрасывают следующие исключения:

1.Оба конструктора класса выбрасывают
исключение IllegalArgumentException, если левая граница области определения больше или равна правой, а также если предлагаемое количество точек меньше двух.

2.Методы getPoint(), setPoint(), getPointX(), setPointX(), getPointY(), setPointY( ) и deletePoint() выбрасывают
исключение FunctionPointIndexOutOfBoundsException, если переданный в метод номер выходит за границы набора точек.

3. Методы setPoint() и setPointX() выбрасывают
исключение InappropriateFunctionPointException в том случае, если координата x задаваемой точки лежит вне интервала, определяемого значениями соседних точек табулированной функции.

4. Метод addPoint() выбрасывает
исключение InappropriateFunctionPointException, если в наборе точек функции есть точка, абсцисса которой совпадает с абсциссой добавляемой точки.

5. Метод deletePoint() выбрасывает исключение IllegalStateException, если на момент удаления точки количество точек в наборе менее трех.

```java
public ArrayTabulatedFunction(double leftX, double rightX, int pointsCount) {

    if (pointsCount < 2) {
        throw new IllegalArgumentException("Points count must be at least 2, got: " + pointsCount);
    }
    if (rightX - leftX < EPSILON) {
        throw new IllegalArgumentException(
                "Left border must be less than right border. Got: leftX=" + leftX + ", rightX=" + rightX
        );
    }

    this.pointsCount = pointsCount;
    this.points = new FunctionPoint[pointsCount + 10];

    double step = (rightX - leftX) / (pointsCount - 1);
    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        points[i] = new FunctionPoint(x, 0.0);
    }
}

public ArrayTabulatedFunction(double leftX, double rightX, double[] values) {
    if (values.length < 2) {
        throw new IllegalArgumentException("Values array must have at least 2 elements");
    }
    if (rightX - leftX < EPSILON) {
        throw new IllegalArgumentException(
                "Left border must be less than right border. Got: leftX=" + leftX + ", rightX=" + rightX
        );
    }

    this.pointsCount = values.length;
    this.points = new FunctionPoint[pointsCount + 10];
```

Код 2.1

```java
public FunctionPoint getPoint(int index) {
    checkIndex(index);
    return new FunctionPoint(points[index]);
}

public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {
    checkIndex(index);

    double newX = point.getX();
    double leftBound = (index > 0) ? points[index - 1].getX() : -Double.MAX_VALUE;
    double rightBound = (index < pointsCount - 1) ? points[index + 1].getX() : Double.MAX_VALUE;

    if (newX <= leftBound + EPSILON || newX >= rightBound - EPSILON) {
        throw new InappropriateFunctionPointException(
                "New X coordinate " + newX + " would break point ordering. " +
                        "Must be in (" + leftBound + ", " + rightBound + ")"
        );
    }

    points[index] = new FunctionPoint(point);
}

public double getPointX(int index) {
    checkIndex(index);
    return points[index].getX();
}
```

Код 2.2

```
public void setPointX(int index, double x) throws InappropriateFunctionPointException {
    checkIndex(index);

    double leftBound = (index > 0) ? points[index - 1].getX() : -Double.MAX_VALUE;
    double rightBound = (index < pointsCount - 1) ? points[index + 1].getX() : Double.MAX_VALUE;

    if (x <= leftBound + EPSILON || x >= rightBound - EPSILON) {
        throw new InappropriateFunctionPointException(
                "New X coordinate " + x + " would break point ordering. " +
                        "Must be in (" + leftBound + ", " + rightBound + ")"
        );
    }

    points[index].setX(x);
}

public double getPointY(int index) {
    checkIndex(index);
    return points[index].getY();
}

public void setPointY(int index, double y) {
    checkIndex(index);
    points[index].setY(y);
}
```

Код 2.3

```
public void deletePoint(int index) {
    if (pointsCount <= 2) {
        throw new IllegalStateException(
                "Cannot delete point: function must have at least 2 points. Current: " + pointsCount
        );
    }

    checkIndex(index);

    System.arraycopy(points, index + 1, points, index, pointsCount - index - 1);
    pointsCount--;
}

public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {

    for (int i = 0; i < pointsCount; i++) {
        if (Math.abs(points[i].getX() - point.getX()) < EPSILON) {
            throw new InappropriateFunctionPointException(
                    "Point with X = " + point.getX() + " already exists at index " + i
            );
        }
    }
    if (pointsCount == points.length) {
        FunctionPoint[] newArray = new FunctionPoint[points.length * 2];
        System.arraycopy(points, 0, newArray, 0, pointsCount);
```

Код 2.4

```
private void checkIndex(int index) {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(
                "Index " + index + " is out of bounds [0, " + (pointsCount - 1) + "]"
        );
    }
}
```

Код 2.5 (вспомогательный метод)

Задание 4: В пакете functions создан класс LinkedListTabulatedFunction, объект которого также описывает табулированную функцию, но для хранения набора точек в нем используется не массив, а связный список. Этот класс совмещает в себе две функции: с одной стороны, он описывает связный список и работу с ним, а с другой стороны, он описывает работу с табулированной функцией и ее точками. Для реализации первой функции реализованы:
FunctionNode (класс элементов списка, содержащий информационное поле для хранения данных типа FunctionPoint, а также поля для хранения ссылок на предыдущий и следующий элемент), метод getNodeByIndex (возвращающий ссылку на объект элемента списка по его номеру), метод addNodeToTail (добавляющий новый элемент в конец списка и возвращающий ссылку на объект этого элемента), метод addNodeByIndex (добавляющий новый элемент в указанную позицию списка и возвращающий ссылку на объект этого элемента), метод deleteNodeByIndex (удаляющий элемент списка по номеру и возвращающий ссылку на объект удаленного элемента).

```java
package functions;

public class LinkedListTabulatedFunction {
    protected class FunctionNode {
        FunctionPoint point;
        FunctionNode prev;
        FunctionNode next;

        FunctionNode(FunctionPoint point, FunctionNode prev, FunctionNode next) {
            this.point = point;
            this.prev = prev;
            this.next = next;
        }

        FunctionNode() {
            this.point = null;
            this.prev = this;
            this.next = this;
        }
    }

    private FunctionNode head;
    private int pointsCount;
    private FunctionNode lastAccessedNode;
    private int lastAccessedIndex;

    public LinkedListTabulatedFunction() {
        head = new FunctionNode();
        head.prev = head;
        head.next = head;
        pointsCount = 0;
        lastAccessedNode = head;
        lastAccessedIndex = -1;
    }
```

Код 3.1

```
private FunctionNode getNodeByIndex(int index) {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException();
    }


    int startIndex;
    FunctionNode current;

    if (Math.abs(index - lastAccessedIndex) < Math.abs(index)) {
        startIndex = lastAccessedIndex;
        current = lastAccessedNode;
    } else {
        startIndex = 0;
        current = head.next;
    }

    if (index > startIndex) {
        for (int i = startIndex; i < index; i++) {
            current = current.next;
        }
    } else {
        for (int i = startIndex; i > index; i--) {
            current = current.prev;
        }
    }

    lastAccessedNode = current;
    lastAccessedIndex = index;
    return current;
}
```

Код 3.2

```
private FunctionNode addNodeToTail() {
    FunctionNode newNode = new FunctionNode(new FunctionPoint(), head.prev, head);
    head.prev.next = newNode;
    head.prev = newNode;
    pointsCount++;
    return newNode;
}

private FunctionNode addNodeByIndex(int index) {
    FunctionNode nodeAtIndex = getNodeByIndex(index);
    FunctionNode newNode = new FunctionNode(new FunctionPoint(), nodeAtIndex.prev, nodeAtIndex);
    nodeAtIndex.prev.next = newNode;
    nodeAtIndex.prev = newNode;
    pointsCount++;
    lastAccessedNode = newNode;
    lastAccessedIndex = index;
    return newNode;
}
```

Код 3.3

```
private FunctionNode deleteNodeByIndex(int index) {
    if (pointsCount <= 2) {
        throw new IllegalStateException("Cannot delete point - minimum 2 points required");
    }

    FunctionNode nodeToDelete = getNodeByIndex(index);
    nodeToDelete.prev.next = nodeToDelete.next;
    nodeToDelete.next.prev = nodeToDelete.prev;
    pointsCount--;

    if (lastAccessedNode == nodeToDelete) {
        lastAccessedNode = nodeToDelete.next;
        if (lastAccessedNode == head) {
            lastAccessedNode = head.next;
            lastAccessedIndex = 0;
        }
    } else if (lastAccessedIndex > index) {
        lastAccessedIndex--;
    }

    return nodeToDelete;
}
```

Код 3.4

Задание 5: Для обеспечения второй функции класса LinkedListTabulatedFunction реализованы в классе конструкторы и методы, аналогичные конструкторам и методам класса TabulatedFunction

```
110     public LinkedListTabulatedFunction(double leftX, double rightX, int pointsCount) {
111         this();
112         if (leftX >= rightX) {
113             throw new IllegalArgumentException("Left border must be less than right border");
114         }
115         if (pointsCount < 2) {
116             throw new IllegalArgumentException("Points count must be at least 2");
117         }
118         double step = (rightX - leftX) / (pointsCount - 1);
119         for (int i = 0; i < pointsCount; i++) {
120             double x = leftX + i * step;
121             try {
122                 addPoint(new FunctionPoint(x, 0.0));
123             } catch (InappropriateFunctionPointException e) {
124                 throw new RuntimeException("Unexpected error in constructor", e);
125             }
126         }
127     }
```

Код 4.1

```
128     public LinkedListTabulatedFunction(double leftX, double rightX, double[] values) {
129         this();
130         if (leftX >= rightX) {
131             throw new IllegalArgumentException("Left border must be less than right border");
132         }
133         if (values.length < 2) {
134             throw new IllegalArgumentException("Points count must be at least 2");
135         }
136 
137         double step = (rightX - leftX) / (values.length - 1);
138         for (int i = 0; i < values.length; i++) {
139             double x = leftX + i * step;
140             try {
141                 addPoint(new FunctionPoint(x, values[i]));
142             } catch (InappropriateFunctionPointException e) {
143                 throw new RuntimeException("Unexpected error in constructor", e);
144             }
145         }
146     }
147 
148     public double getLeftDomainBorder() {
149         if (pointsCount == 0) {
150             throw new IllegalStateException("Function has no points");
151         }
152         return head.next.point.getX();
153     }
154 
155     public double getRightDomainBorder() {
156         if (pointsCount == 0) {
157             throw new IllegalStateException("Function has no points");
158         }
159         return head.prev.point.getX();
160     }
```

Код 4.2

```
163     public int getPointsCount() {
164         return pointsCount;
165     }
166     public FunctionPoint getPoint(int index) {
167         checkIndex(index);
168 
169         return new FunctionPoint(getNodeByIndex(index).point);
170     }
171     public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {
172         checkIndex(index);
173         FunctionNode node = getNodeByIndex(index);
174 
175         double newX = point.getX();
176         double leftBound = (index > 0) ? node.prev.point.getX() : -Double.MAX_VALUE;
177         double rightBound = (index < pointsCount - 1) ? node.next.point.getX() : Double.MAX_VALUE;
178 
179         if (newX <= leftBound + EPSILON || newX >= rightBound - EPSILON) {
180             throw new InappropriateFunctionPointException("X coordinate violates ordering");
181         }
182 
183         FunctionNode current = head.next;
184         while (current != head) {
185             if (current != node && Math.abs(current.point.getX() - newX) < EPSILON) {
186                 throw new InappropriateFunctionPointException("Point with same X already exists");
187             }
188             current = current.next;
189         }
190         node.point = new FunctionPoint(point);
191     }
```

Код 4.3

```
194     public double getPointX(int index) {
195         checkIndex(index);
196         return getNodeByIndex(index).point.getX();
197     }
198 
199 
200     public void setPointX(int index, double x) throws InappropriateFunctionPointException {
201         checkIndex(index);
202         FunctionNode node = getNodeByIndex(index);
203 
204         double leftBound = (index > 0) ? node.prev.point.getX() : -Double.MAX_VALUE;
205         double rightBound = (index < pointsCount - 1) ? node.next.point.getX() : Double.MAX_VALUE;
206 
207         if (x <= leftBound + EPSILON || x >= rightBound - EPSILON) {
208             throw new InappropriateFunctionPointException("X coordinate violates ordering");
209         }
210 
211         FunctionNode current = head.next;
212         while (current != head) {
213             if (current != node && Math.abs(current.point.getX() - x) < EPSILON) {
214                 throw new InappropriateFunctionPointException("Point with same X already exists");
215             }
216             current = current.next;
217         }
218 
219         node.point.setX(x);
220     }
```

Код 4.4

```
222     public double getPointY(int index) {
223         checkIndex(index);
224         return getNodeByIndex(index).point.getY();
225     }
226 
227     public void setPointY(int index, double y) {
228         checkIndex(index);
229         getNodeByIndex(index).point.setY(y);
230     }
231 
232     private void checkIndex(int index) {
233         if (index < 0 || index >= pointsCount) {
234             throw new FunctionPointIndexOutOfBoundsException("Index: " + index);
235         }
236     }
237     public void deletePoint(int index) {
238         checkIndex(index);
239         if (pointsCount <= 2) {
240             throw new IllegalStateException("Cannot delete point - minimum 2 points required");
241         }
242 
243         deleteNodeByIndex(index);
244     }
245 
```

Код 4.5

```
247     public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {
248         FunctionNode current = head.next;
249         while (current != head && current.point != null) {
250             if (Math.abs(current.point.getX() - point.getX()) < EPSILON) {
251                 throw new InappropriateFunctionPointException("Point with same X already exists");
252             }
253             current = current.next;
254         }
255         current = head.next;
256         int index = 0;
257         while (current != head && current.point != null && current.point.getX() < point.getX()) {
258             current = current.next;
259             index++;
260         }
261         FunctionNode newNode = new FunctionNode(new FunctionPoint(point), null, null);
262         if (pointsCount == 0) {
263             newNode.prev = head;
264             newNode.next = head;
265             head.next = newNode;
266             head.prev = newNode;
267         } else if (index == pointsCount) {
268             newNode.prev = head.prev;
269             newNode.next = head;
270             head.prev.next = newNode;
271             head.prev = newNode;
272         } else {
273             newNode.prev = current.prev;
274             newNode.next = current;
275             current.prev.next = newNode;
276             current.prev = newNode;
277         }
278         pointsCount++;
279         lastAccessedNode = head;
280         lastAccessedIndex = -1;
```

Код 4.6

```
283     public double getFunctionValue(double x) {
284         if (pointsCount == 0) {
285             return Double.NaN;
286         }
287         double left = getLeftDomainBorder();
288         double right = getRightDomainBorder();
289         if (x < left - EPSILON || x > right + EPSILON) {
290             return Double.NaN;
291         }
292         FunctionNode current = head.next;
293         while (current != head) {
294             if (Math.abs(current.point.getX() - x) < EPSILON) {
295                 return current.point.getY();
296             }
297             current = current.next;
298         }
299         current = head.next;
300         while (current.next != head) {
301             double x1 = current.point.getX();
302             double x2 = current.next.point.getX();
303             if (x >= x1 - EPSILON && x <= x2 + EPSILON) {
304                 double y1 = current.point.getY();
305                 double y2 = current.next.point.getY();
306                 return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
307             }
308             current = current.next;
309         }
310         return Double.NaN;
311     }
```

Код 4.7

Задание 6: Класс TabulatedFunction переименован в класс ArrayTabulatedFunction. Создан интерфейс TabulatedFunction, содержащий объявления общих методов классов ArrayTabulatedFunction и LinkedListTabulatedFunction, оба класса функций реализовывают интерфейс.

```java
package functions;

public interface TabulatedFunction {
    double getLeftDomainBorder();
    double getRightDomainBorder();
    double getFunctionValue(double x);
    int getPointsCount();
    FunctionPoint getPoint(int index);
    void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException;
    double getPointX(int index);
    void setPointX(int index, double x) throws InappropriateFunctionPointException;
    double getPointY(int index);
    void setPointY(int index, double y);
    void deletePoint(int index);
    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;
    String toString();
}
```

Код 5.1

```java
public class ArrayTabulatedFunction implements TabulatedFunction{
```

Код 5.2

```java
public class LinkedListTabulatedFunction implements TabulatedFunction{
```

Код 5.3

Задание 7: создан класс Main, с помощью которого будут проверяться работа написанных классов, и выведен результат его выполнения в консоль.

```java
import functions.*;

public class Main {
    public static void main(String[] args) {
        try {
            System.out.println(" Testing ArrayTabulatedFunction ");
            TabulatedFunction arrayFunc = new ArrayTabulatedFunction(0, 10, 5);
            testFunction(arrayFunc);
            System.out.println("\n Testing LinkedListTabulatedFunction ");
            TabulatedFunction listFunc = new LinkedListTabulatedFunction(0, 10, 5);
            testFunction(listFunc);
            testAllExceptions();
        } catch (Exception e) {
            System.out.println("Exception: " + e.getClass().getName() + " - " + e.getMessage());
        }
    }
```

Код 6.1

```
    private static void testFunction(TabulatedFunction func) {
        System.out.println("\n Initial function state ");
        System.out.println("Type: " + func.getClass().getSimpleName());
        System.out.println("Left border: " + func.getLeftDomainBorder());
        System.out.println("Right border: " + func.getRightDomainBorder());
        System.out.println("Points count: " + func.getPointsCount());
        System.out.println("Initial points:");
        for (int i = 0; i < func.getPointsCount(); i++) {
            FunctionPoint point = func.getPoint(i);
            System.out.println("  Point[" + i + "]: (" + point.getX() + ", " + point.getY() + ")");
        }
        System.out.println("\n Testing function values ");
        double testX = 5.0;
        System.out.println("Value at x=" + testX + ": " + func.getFunctionValue(testX));
        System.out.println("Value at x=-1: " + func.getFunctionValue(-1));
        System.out.println("Value at x=11: " + func.getFunctionValue(11));
        System.out.println("\n Testing POINT ADDITION ");
        System.out.println("\n1. Adding point to the END:");
        FunctionPoint newPoint1 = new FunctionPoint(12, 24);
        System.out.println("Adding: (" + newPoint1.getX() + ", " + newPoint1.getY() + ")");
        try {
            func.addPoint(newPoint1);
            System.out.println("Points count after addition: " + func.getPointsCount());
            System.out.println("Last point: (" + func.getPoint(func.getPointsCount() - 1).getX() +
                    ", " + func.getPoint(func.getPointsCount() - 1).getY() + ")");
        } catch (InappropriateFunctionPointException e) {
            System.out.println("Caught InappropriateFunctionPointException: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Caught exception: " + e.getClass().getName() + " - " + e.getMessage());
        }
        System.out.println("\n2. Note: insertPoint method might not be available in TabulatedFunction interface");
        System.out.println("   Testing available methods instead:");
        System.out.println("\nAll points after addition:");
```

Код 6.2

```
        System.out.println("\nAll points after addition:");
        for (int i = 0; i < func.getPointsCount(); i++) {
            FunctionPoint point = func.getPoint(i);
            System.out.println("  Point[" + i + "]: (" + point.getX() + ", " + point.getY() + ")");
        }
        System.out.println("\n Testing POINT DELETION ");
        try {
            System.out.println("\n1. Deleting point at index 0:");
            FunctionPoint firstPoint = func.getPoint(0);
            System.out.println("Deleting point: (" + firstPoint.getX() + ", " + firstPoint.getY() + ")");
            func.deletePoint(0);
            System.out.println("Points count after deletion: " + func.getPointsCount());
            System.out.println("New first point (index 0): (" + func.getPoint(0).getX() +
                    ", " + func.getPoint(0).getY() + ")");
        } catch (Exception e) {
            System.out.println("deletePoint method not available or error: " + e.getMessage());
        }
        System.out.println("\nAll points after operations:");
        for (int i = 0; i < func.getPointsCount(); i++) {
            FunctionPoint point = func.getPoint(i);
            System.out.println("  Point[" + i + "]: (" + point.getX() + ", " + point.getY() + ")");
        }
        System.out.println("\n Testing POINT REPLACEMENT ");
        try {
            System.out.println("\n1. Replacing point at index 0:");
            FunctionPoint oldPoint0 = func.getPoint(0);
            FunctionPoint newPoint0 = new FunctionPoint(0, 100);
            System.out.println("Old point: (" + oldPoint0.getX() + ", " + oldPoint0.getY() + ")");
            System.out.println("New point: (" + newPoint0.getX() + ", " + newPoint0.getY() + ")");
            func.setPoint(0, newPoint0);
            System.out.println("After replacement - Point[0]: (" + func.getPoint(0).getX() +
                    ", " + func.getPoint(0).getY() + ")");
        } catch (Exception e) {
```

Код 6.3

```
            System.out.println("setPoint method not available or error: " + e.getMessage());
        }
        System.out.println("\n--- Final function state ---");
        System.out.println("Points count: " + func.getPointsCount());
        System.out.println("Left border: " + func.getLeftDomainBorder());
        System.out.println("Right border: " + func.getRightDomainBorder());
        System.out.println("\nVerifying X values are sorted:");
        boolean sorted = true;
        for (int i = 1; i < func.getPointsCount(); i++) {
            double x1 = func.getPoint(i - 1).getX();
            double x2 = func.getPoint(i).getX();
            if (x1 >= x2) {
                System.out.println("  ERROR: x[" + (i - 1) + "] = " + x1 + " >= x[" + i + "] = " + x2);
                sorted = false;
            }
        }
        if (sorted) {
            System.out.println("   All X values are properly sorted");
        }
    }
    private static void testAllExceptions() {
        System.out.println("\n Testing all exceptions and special cases ");
        System.out.println("\n1. Constructor exceptions:");
        try {
            System.out.println("  a) Creating function with left > right:");
            TabulatedFunction func = new ArrayTabulatedFunction(10, 0, 5);
        } catch (IllegalArgumentException e) {
            System.out.println("     Caught IllegalArgumentException: " + e.getMessage());
        }
        try {
            System.out.println("  b) Creating function with < 2 points:");
            TabulatedFunction func = new ArrayTabulatedFunction(0, 10, 1);
        } catch (IllegalArgumentException e) {
            System.out.println("     Caught IllegalArgumentException: " + e.getMessage());
```

Код 6.4

```
        }
        System.out.println("\n2. Index out of bounds exceptions:");
        try {
            System.out.println("  a) Accessing out of bounds index (getPoint):");
            TabulatedFunction func = new ArrayTabulatedFunction(0, 10, 3);
            func.getPoint(5);
        } catch (FunctionPointIndexOutOfBoundsException e) {
            System.out.println("     Caught FunctionPointIndexOutOfBoundsException: " + e.getMessage());
        }
        try {
            System.out.println("  b) Deleting at invalid index:");
            TabulatedFunction func = new ArrayTabulatedFunction(0, 10, 3);
            func.deletePoint(5);
        } catch (FunctionPointIndexOutOfBoundsException e) {
            System.out.println("     Caught FunctionPointIndexOutOfBoundsException: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("     deletePoint method might not be available");
        }
        try {
            System.out.println("  c) Replacing at invalid index:");
            TabulatedFunction func = new ArrayTabulatedFunction(0, 10, 3);
            func.setPoint(5, new FunctionPoint(15, 30));
        } catch (FunctionPointIndexOutOfBoundsException e) {
            System.out.println("     Caught FunctionPointIndexOutOfBoundsException: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("     setPoint method might not be available");
        }
        System.out.println("\n3. Logic exceptions:");
        try {
            System.out.println("  a) Deleting when only 2 points remain:");
            TabulatedFunction func = new ArrayTabulatedFunction(0, 10, 3);
            func.deletePoint(1);
            func.deletePoint(0);
        } catch (IllegalStateException e) {
```

Код 6.5

```
            System.out.println("     Caught IllegalStateException: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("     deletePoint method might not be available");
        }
        System.out.println("\n4. Duplicate x values exceptions:");
        try {
            System.out.println("  a) Adding point with duplicate X:");
            TabulatedFunction func = new ArrayTabulatedFunction(0, 10, 3);
            func.addPoint(new FunctionPoint(5, 100));
        } catch (InappropriateFunctionPointException e) {
            System.out.println("     Caught InappropriateFunctionPointException: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("     Caught exception: " + e.getClass().getName() + " - " + e.getMessage());
        }
        try {
            System.out.println("  b) Replacing point with X that breaks sorting:");
            TabulatedFunction func = new ArrayTabulatedFunction(0, 10, 5);
            FunctionPoint point1 = func.getPoint(1);
            FunctionPoint point3 = func.getPoint(3);
            func.setPoint(2, new FunctionPoint(point1.getX() - 1, 50));
        } catch (InappropriateFunctionPointException e) {
            System.out.println("     Caught InappropriateFunctionPointException: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("     Caught exception: " + e.getClass().getName() + " - " + e.getMessage());
        }
        System.out.println("\n5. Special cases:");
        try {
            System.out.println("  a) Adding point at the same X as boundary:");
            TabulatedFunction func = new ArrayTabulatedFunction(0, 10, 3);
            func.addPoint(new FunctionPoint(0, 50));
        } catch (InappropriateFunctionPointException e) {
            System.out.println("     Caught InappropriateFunctionPointException: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("     Caught exception: " + e.getClass().getName() + " - " + e.getMessage());
```

Код 6.6

```
        }
        try {
            System.out.println("  b) Replacing with same X but different Y:");
            TabulatedFunction func = new ArrayTabulatedFunction(0, 10, 5);
            FunctionPoint point2 = func.getPoint(2);
            func.setPoint(2, new FunctionPoint(point2.getX(), point2.getY() + 10));
            System.out.println("     Allowed: Same X with different Y");
        } catch (Exception e) {
            System.out.println("     Unexpected exception: " + e.getClass().getName());
        }
        try {
            System.out.println("  c) Testing with negative index:");
            TabulatedFunction func = new ArrayTabulatedFunction(0, 10, 3);
            func.getPoint(-1);
        } catch (FunctionPointIndexOutOfBoundsException e) {
            System.out.println("     Caught FunctionPointIndexOutOfBoundsException for negative index");
        }
        System.out.println("\n ALL EXCEPTION TESTS COMPLETED ");
    }
}
```

Код 6.7

```
PS C:\Javalabs\lab3> javac functions/*.java
PS C:\Javalabs\lab3> javac Main.java
PS C:\Javalabs\lab3> java Main
 Testing ArrayTabulatedFunction

 Initial function state
Type: ArrayTabulatedFunction
Left border: 0.0
Right border: 10.0
Points count: 5
Initial points:
  Point[0]: (0.0, 0.0)
  Point[1]: (2.5, 0.0)
  Point[2]: (5.0, 0.0)
  Point[3]: (7.5, 0.0)
  Point[4]: (10.0, 0.0)

 Testing function values
Value at x=5.0: 0.0
Value at x=-1: NaN
Value at x=11: NaN

 Testing POINT ADDITION

1. Adding point to the END:
Adding: (12.0, 24.0)
Points count after addition: 6
Last point: (12.0, 24.0)

2. Note: insertPoint method might not be available in TabulatedFunction interface
   Testing available methods instead:

All points after addition:
  Point[0]: (0.0, 0.0)
  Point[1]: (2.5, 0.0)
  Point[2]: (5.0, 0.0)
  Point[3]: (7.5, 0.0)
  Point[4]: (10.0, 0.0)
  Point[5]: (12.0, 24.0)

 Testing POINT DELETION
```

Реализация 6.1

```
 Testing POINT DELETION

1. Deleting point at index 0:
Deleting point: (0.0, 0.0)
Points count after deletion: 5
New first point (index 0): (2.5, 0.0)

All points after operations:
  Point[0]: (2.5, 0.0)
  Point[1]: (5.0, 0.0)
  Point[2]: (7.5, 0.0)
  Point[3]: (10.0, 0.0)
  Point[4]: (12.0, 24.0)

 Testing POINT REPLACEMENT

1. Replacing point at index 0:
Old point: (2.5, 0.0)
New point: (0.0, 100.0)
After replacement - Point[0]: (0.0, 100.0)

--- Final function state ---
Points count: 5
Left border: 0.0
Right border: 12.0

Verifying X values are sorted:
   All X values are properly sorted

 Testing LinkedListTabulatedFunction

 Initial function state
Type: LinkedListTabulatedFunction
Left border: 0.0
Right border: 10.0
Points count: 5
Initial points:
  Point[0]: (0.0, 0.0)
  Point[1]: (2.5, 0.0)
  Point[2]: (5.0, 0.0)
```

Реализация 6.2

```
  Point[2]: (5.0, 0.0)
  Point[3]: (7.5, 0.0)
  Point[4]: (10.0, 0.0)

 Testing function values
Value at x=5.0: 0.0
Value at x=-1: NaN
Value at x=11: NaN

 Testing POINT ADDITION

1. Adding point to the END:
Adding: (12.0, 24.0)
Points count after addition: 6
Last point: (12.0, 24.0)

2. Note: insertPoint method might not be available in TabulatedFunction interface
   Testing available methods instead:

All points after addition:
  Point[0]: (0.0, 0.0)
  Point[1]: (2.5, 0.0)
  Point[2]: (5.0, 0.0)
  Point[3]: (7.5, 0.0)
  Point[4]: (10.0, 0.0)
  Point[5]: (12.0, 24.0)

 Testing POINT DELETION

1. Deleting point at index 0:
Deleting point: (0.0, 0.0)
Points count after deletion: 5
New first point (index 0): (2.5, 0.0)

All points after operations:
  Point[0]: (2.5, 0.0)
  Point[1]: (5.0, 0.0)
  Point[2]: (7.5, 0.0)
  Point[3]: (10.0, 0.0)
  Point[4]: (12.0, 24.0)
```

Реализация 6.3

```
 Testing POINT REPLACEMENT

1. Replacing point at index 0:
Old point: (2.5, 0.0)
New point: (0.0, 100.0)
After replacement - Point[0]: (0.0, 100.0)

--- Final function state ---
Points count: 5
Left border: 0.0
Right border: 12.0

Verifying X values are sorted:
   All X values are properly sorted

 Testing all exceptions and special cases

1. Constructor exceptions:
  a) Creating function with left > right:
     Caught IllegalArgumentException: Left border must be less than right border. Got: leftX=10.0, rightX=0.0
  b) Creating function with < 2 points:
     Caught IllegalArgumentException: Points count must be at least 2, got: 1

2. Index out of bounds exceptions:
  a) Accessing out of bounds index (getPoint):
     Caught FunctionPointIndexOutOfBoundsException: Index 5 is out of bounds [0, 2]
  b) Deleting at invalid index:
     Caught FunctionPointIndexOutOfBoundsException: Index 5 is out of bounds [0, 2]
  c) Replacing at invalid index:
     Caught FunctionPointIndexOutOfBoundsException: Index 5 is out of bounds [0, 2]

3. Logic exceptions:
  a) Deleting when only 2 points remain:
     Caught IllegalStateException: Cannot delete point: function must have at least 2 points. Current: 2

4. Duplicate x values exceptions:
  a) Adding point with duplicate X:
     Caught InappropriateFunctionPointException: Point with X = 5.0 already exists at index 1
  b) Replacing point with X that breaks sorting:
     Caught InappropriateFunctionPointException: New X coordinate 1.5 would break point ordering. Must be in (2.5, 7.5)
```

Реализация 6.4

```
4. Duplicate x values exceptions:
  a) Adding point with duplicate X:
     Caught InappropriateFunctionPointException: Point with X = 5.0 already exists at index 1
  b) Replacing point with X that breaks sorting:
     Caught InappropriateFunctionPointException: New X coordinate 1.5 would break point ordering. Must be in (2.5, 7.5)

5. Special cases:
  a) Adding point at the same X as boundary:
     Caught InappropriateFunctionPointException: Point with X = 0.0 already exists at index 0
  b) Replacing with same X but different Y:
     Allowed: Same X with different Y
  c) Testing with negative index:
     Caught FunctionPointIndexOutOfBoundsException for negative index

 ALL EXCEPTION TESTS COMPLETED
PS C:\Javalabs\lab3>
```

Реализация 6.5